

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Clase 10: Archivos CSV

Archivos



Un **archivo** (en inglés, *file*) es una colección numerada y ordenada de bytes, que almacena datos procesables por un programa.

Se almacena en un **medio físico** (disco rígido, pendrive, etc.).



Un **programa en ejecución** puede leer y/o escribir datos almacenados en distintos archivos.

A su vez, las variables de un programa en ejecución viven en la **memoria RAM** de la computadora, mucho más rápida que un disco rígido.

Archivos

En el nombre del archivo suele indicarse, mediante la *extensión*, la forma en que se organiza el contenido del archivo.



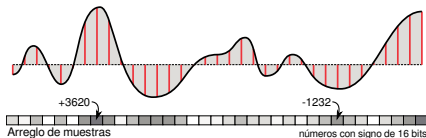
Según su contenido, existen dos tipos genéricos de archivos:

- ▶ **Archivos binarios**, que organizan su contenido de distintas maneras (.png, .wav, .mp4, .pdf, .xlsx, .docx, por ejemplo).
- ▶ **Archivos de texto**, caracterizados por contener únicamente caracteres imprimibles (.txt, .py, .html, .xml, .csv, .tex, por ejemplo).

Por simplicidad, vamos a trabajar con **archivos de texto**, pero no hay mayores diferencias conceptuales con los archivos binarios.

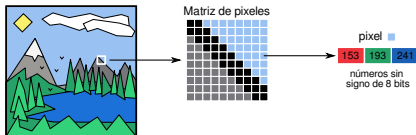
Algunos tipos de archivos binarios

Sonido



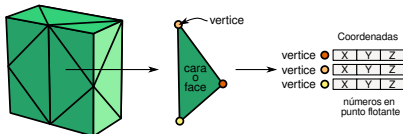
Ejemplo: Formato WAV
Almacena muestras de señales de audio sin comprimir, permite (p.ej.) frecuencias de muestro de 16 bits a 44kHz.

Imagen



Ejemplo: Formato BMP
Guarda mapas de bits sin compresión. Permite guardar imágenes en escala de grises y en colores de 24 o 32 bits.

Diseño 3D



Ejemplo: Formato STL
Permite representar superficies 3D por medio de la descripción de triángulos en coordenadas cartesianas.

Archivos de texto

```
def sumar_divisores(n):  
    suma = 0  
    i = 1  
    while divisor <= n:  
        if n % i == 0:  
            suma = suma + i  
        i = i + 1  
    return suma
```

suma.py

```
Lorem ipsum dolor sit amet,  
consectetur adipiscing elit.  
Fusce interdum pretium ante,  
nec lobortis erat venenatis  
nec. Nullam auctor lectus at  
enim consequat, non  
tincidunt risus ullamcorper.  
Suspendisse potenti.
```

lorem.txt

- ▶ Se compone de **caracteres** codificados en un **sistema de codificación** (ASCII, UTF-8, ISO-8859-1, etc.), que representan letras, números, símbolos y espacios.
- ▶ Los archivos de texto pueden ser creados y editados con cualquier editor de texto (Notepad, Atom, Vim, Sublime, etc.)

Archivos CSV

Los **archivos CSV** (*Comma Separated Values*) son archivos de texto con un formato común para almacenar datos.

```
Nombre,Edad,Profesión,Nacionalidad
Juan,25,Ingeniero,Argentina
María,30,Abogada,Mexicana
Ana,35,Médica,Colombiana
Luis,32,Arquitecto,Peruana
Sofía,27,Periodista,Chilena
Carlos,29,Ingeniero,Brasileña
Laura,31,Economista,Paraguaya
Lucía,29,Contadora,Argentina
Jorge,33,Profesor,Argentina
Roberto,35,Abogado,Uruguay
Marcela,31,Arquitecta,Uruguay
```

- ▶ Cada fila representa un registro o una entrada de datos, y las columnas representan los campos o atributos.
- ▶ Los valores en un archivo CSV están separados por comas (",") o por otro delimitador (por ejemplo, punto y coma).

Lectura de archivos de texto

Con `open(filename)` abrimos un archivo en `modo lectura`.

```
f = open('padron.csv')
```

Nota: En Windows, puede ser necesario un argumento adicional para reconocer la codificación del texto: `f = open('padron.csv', encoding='utf-8')`

Se puede leer todo el contenido del archivo con `f.read()` (devuelve un `str`):

```
texto = f.read() # Cuidado: ¡puede consumir mucha memoria!  
print(texto)
```

También se pueden leer las líneas de a una con `f.readline()` (devuelve un `str`):

```
print(f.readline()) # imprime 'Nombre,Edad,Profesión , Nacionalidad '  
print(f.readline()) # imprime 'Juan,25,Ingeniero , Argentina '
```

Forma recomendada: `iterar` el archivo, leyendo línea a línea:

```
for linea in f:  
    print(linea)
```

Lectura de archivos de texto

```
f = open('padron.csv')  
for linea in f:  
    print(linea)
```

Una forma equivalente de hacer lo mismo:

```
with open('padron.csv') as f:  
    for linea in f:  
        print(linea)
```

Escritura de archivos de texto

Con `open(filename, 'w')` abrimos un archivo en **modo escritura** (w es por *write*).

```
f = open('primeros-diez.txt', 'w')
```

Escribimos con la instrucción `f.write('...')`.

Ejemplo: se escriben los primeros diez números naturales en el archivo `primeros-diez.txt`, uno por línea.

```
for i in [1,2,3,4,5,6,7,8,9,10]:  
    f.write(str(i) + '\n')    # \n indica fin de línea
```

Al terminar, es buena costumbre **cerrar** el archivo:

```
f.close()
```

Escritura de archivos de texto

```
f = open('primeros-diez.txt', 'w')
for i in [1,2,3,4,5,6,7,8,9,10]:
    f.write(str(i) + '\n')    # \n indica fin de línea
f.close()
```

Una forma equivalente de hacer lo mismo:

```
with open('primeros-diez.txt', 'w') as f:
    for i in [1,2,3,4,5,6,7,8,9,10]:
        f.write(str(i) + '\n')    # \n indica fin de línea
```

Una vez que concluye la ejecución del bloque de código asociado al `with open(...)` el archivo se cierra automáticamente.

Archivos de texto

Ejercicio. (1) Usando los datos contenidos en el archivo `provincias.csv`¹:

- (a) Completar el programa `ej-1a.py` para que calcule la población total de Argentina.
- (b) Completar el programa `ej-1b.py` para que imprima los nombres de las provincias que tienen más de dos millones de habitantes.

¹Datos tomados de Wikipedia: <https://tinyurl.com/provarg>.

Diccionarios *(ayudamemoria)*

Un diccionario asocia **claves** a **valores**.

- Ejemplos: país/población, depto/inquilino, etc.

```
persona = dict()  
persona['Nombre'] = 'Juan'  
persona['Edad'] = '25'  
persona['Profesión'] = 'Ingeniero'  
persona['Nacionalidad'] = 'Argentina'
```

Biblioteca csv (*lectura*)

Archivo 'padron.csv':

```
Nombre,Edad,Profesión,Nacionalidad
Juan,25,Ingeniero,Argentina
María,30,Abogada,Mexicana
Ana,35,Médica,Colombiana
...
```

```
1  import csv
2
3  filename = 'padron.csv' # nombre del archivo con los datos
4  f = open(filename)
5
6  personas = []
7  dict_reader = csv.DictReader(f)
8  for d in dict_reader:    # d es un diccionario
9      nom  = d['Nombre']
10     ed   = int(d['Edad'])
11     prof = d['Profesión']
12     nac  = d['Nacionalidad']
13     p = Persona(nom,ed,prof,nac)
14     personas.append(p)
```

En la línea 4 abrimos un archivo en modo **lectura**.

Con el **for** de la línea 8, leemos el archivo, una línea por vez.

En cada iteración, **d** es un **diccionario** que tiene como **claves** (de tipo str) los nombres de las columnas (Nombre, Edad, Profesión y Nacionalidad), asociados a los **valores** (de tipo str) correspondientes para cada persona.

Ejemplo: en la primera iteración, **d['Nombre']** → **'Juan'**.

Biblioteca csv (*escritura*)

```
1  import csv
2
3  f = open('clubes-primera.csv', 'w')
4  campos = ['NOMBRE', 'FUNDACIÓN', 'CANCHA']
5  writer = csv.DictWriter(f, fieldnames=campos)
6
7  writer.writeheader()
8  writer.writerow({'NOMBRE': 'Boca Jrs', 'FUNDACIÓN': '1905',
9                  'CANCHA': 'Bombonera'})
10 writer.writerow({'NOMBRE': 'Racing', 'FUNDACIÓN': '1903', 'CANCHA': 'Cilindro'})
11 writer.writerow({'NOMBRE': 'River Plate', 'FUNDACIÓN': '1901',
12                 'CANCHA': 'Monumental'})
13 ...
14
15 f.close()
```

- ▶ Línea 3: abrimos el archivo en modo **escritura**.
- ▶ Línea 4: definimos una lista que contiene los nombres de las columnas.
- ▶ Línea 5: creamos un DictWriter a partir del archivo y la lista de columnas.
- ▶ Línea 7: escribimos los títulos en el archivo.
- ▶ Línea 8: comenzamos a escribir datos en el archivo.
- ▶ Línea 15: cerramos el archivo.

Archivos CSV

Ejercicio. (2)

Usando los datos contenidos en el archivo `provincias.csv`, completar el programa `ej-2.py` para escribir un nuevo archivo llamado `provincias-reducido.csv` que contenga, de cada provincia, únicamente el nombre, el área total y la población. Los datos deben aparecer bajo los títulos `PROVINCIA`, `ÁREA` y `POBLACIÓN`.

A continuación puede observarse un fragmento del archivo que se debe generar.

```
PROVINCIA,ÁREA,POBLACIÓN
Buenos Aires,307571,15625084
CABA,202,2890151
Catamarca,102602,367828
Chaco,99633,1055259
Chubut,224686,509108
Córdoba,165321,3308876
Corrientes,88199,992595
Entre Ríos,78781,1235994
Formosa,72066,530162
Jujuy53219,673307
...
```

Repaso de la clase de hoy

- ▶ Archivos
 - ▶ Binarios y de Texto
 - ▶ Lectura y Escritura
- ▶ Biblioteca csv
 - ▶ DictReader
 - ▶ DictWriter

Bibliografía complementaria:

- ▶ IPP, secciones 13 y 13.1
- ▶ The Python Tutorial, secciones 7.2 y 7.2.1

Con lo visto, ya pueden hacer hasta el ejercicio 10.4 de la guía de ejercicios.